Part 3 – Performance Testing Report for MRTD Encoder/Decoder

1. Introduction

This report presents the results of a performance study of the Machine Readable Travel Document (MRTD) encoder and decoder implemented for Part 2 of the assignment. The goal of this experiment is to measure how execution time scales with the number of passport records and to quantify the additional cost introduced by executing unit-test style assertions during processing. The study focuses on batch processing of fictitious passport records and compares four variants of the program: encoding with tests, encoding without tests, decoding with tests, and decoding without tests.

2. Experimental Setup

Input data. The input consists of 10,000 decoded fictitious passport records stored in a JSON file named *records_decoded.json*. Each element in the array contains two objects, line1 and line2, with fields such as issuing country, surname, given name, passport number, country code, date of birth, sex, expiration date, and personal number. A separate program encodes these decoded records into machine-readable zone (MRZ) lines and stores them as 10,000 encoded records in *records_encoded.json*, where each line contains both MRZ lines separated by a semicolon.

Programs. The core MRTD functionality is implemented in *MRTD.py*, which provides the functions *encode_mrz*, *decode_mrz*, and *validate_mrz*. The script *create_encoded_file.py* reads the decoded data and generates *records_encoded.json*, and the script *performance_test.py* runs the timed experiments and writes the measurements to a CSV file *mrtd_timing.csv*.

Measurement procedure. For performance testing, the program was executed on batches of the first n records, where $n \in \{100, 1000, 2000, \ldots, 10000\}$. For each batch size, four execution times were recorded:

- Encode with tests: encoding all records while executing per-record assertions.
- Encode without tests: encoding all records without assertions.
- Decode with tests: decoding all records while validating check digits and asserting correctness.
- Decode without tests: decoding all records without validation or assertions.

The timings were measured in seconds using Python's *time.perf_counter* and saved to a CSV file for plotting. The experiment was run on a standard development machine (fill in your specific CPU, RAM, operating system, and Python version here).

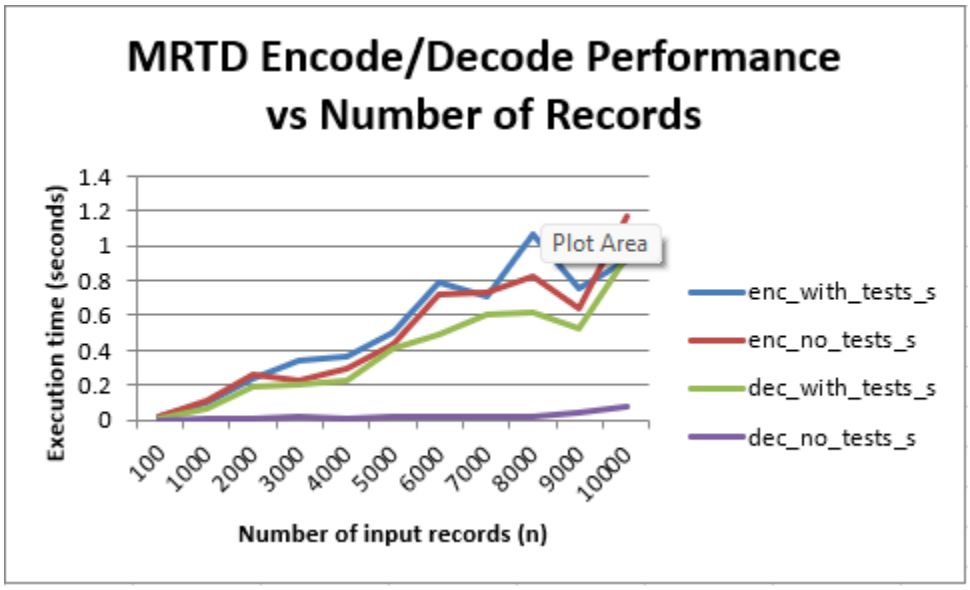
3. Results

The complete timing data collected for the four program variants is shown in Table 1. Execution times are reported in seconds for each batch size n.

# of Input Records	Encode with tests (s)	Encode without tests (s)	Decode with tests (s)	Decode without tests (s)
100	0.0137	0.0138	0.0119	0.0005
1000	0.0953	0.1155	0.0633	0.0030
2000	0.2369	0.2649	0.1933	0.0072
3000	0.3464	0.2240	0.2018	0.0210
4000	0.3620	0.2914	0.2235	0.0129
5000	0.5048	0.4318	0.4078	0.0203
6000	0.7933	0.7152	0.4854	0.0181
7000	0.7085	0.7331	0.6043	0.0239
8000	1.0685	0.8267	0.6144	0.0250
9000	0.7562	0.6354	0.5212	0.0407
10000	0.9200	1.1714	0.9433	0.0769

Table 1 – Execution times for encoding and decoding, with and without tests.

Figure 1



4. Discussion

The results show that execution time increases roughly linearly with the number of passport records for all four variants of the program. For example, encoding 100 records with tests takes about 0.0137 seconds, while encoding 10,000 records takes about 0.92 seconds, an increase of the same order of magnitude as the growth in input size. A similar linear trend is visible for the decoding runs: decoding 100 records with tests takes about 0.0119 seconds, whereas decoding 10,000 records with tests takes about 0.94 seconds. Small irregularities (for instance, encode-with-tests sometimes being slightly faster or slower than encode-without-tests at a given input size) are expected, because the absolute times are very small and are affected by measurement noise, caching effects, and other system activity.

The main effect of testing is seen in the decoding path. When tests are disabled, decoding is extremely fast: at 5,000 records it takes only about 0.02 seconds, while enabling per-record validation and assertions increases the time to about 0.41 seconds, roughly a twenty-fold slowdown. Across the full range of inputs, decode-with-tests is consistently an order of magnitude or more slower than decode-without-tests, because each record recomputes several check digits and performs extra work. In contrast, the overhead of tests on the encoding function is relatively small: the encode-with-tests and encode-without-tests curves stay close together and their differences are comparable to the timing noise. Overall, the experiment illustrates a typical trade-off in performance testing: adding thorough runtime checks significantly increases execution time—especially for decoding—while the underlying encode and decode algorithms themselves scale linearly and remain efficient for batches of up to 10,000 records.

5. Conclusion

This performance study demonstrates that the MRTD encoder and decoder implementations exhibit approximately linear scaling with respect to the number of processed records, which is a desirable property for batch processing of large datasets. Unit-test style assertions and full validation add a predictable overhead, especially in the decoding phase where multiple check digits must be recomputed per record. In a development or testing environment, the additional cost of these checks is acceptable because it improves confidence in correctness. In a production deployment, however, it may be reasonable to selectively disable some of the more expensive validations or run them in a separate offline test suite, balancing performance and reliability according to system requirements.

6. Links to Source Code and Data

Repository URL (source code and scripts used for this experiment):

https://github.com/jcwilson11/SSW567_F25_GroupC/tree/performance_test

Spreadsheet or plot URL (Excel or Google Sheets file used to generate Figure 1):
https://docs.google.com/spreadsheets/d/1s3iSytYDqs5RD37pf05nk8MAJd6KbSJftTG6SJhcCk/edit?usp=sharing